

Theory of Automata

CS-3005

Lecture 2

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Kleene Closure

- Definition: Given an alphabet Σ , we define a language in which any string of letters from Σ is a word, even the null string Λ . We call this language the closure of the alphabet Σ , and denote this language by Σ^* .

Kleene Closure

- Examples:

If $\Sigma = \{ x \}$ then $\Sigma^* = \{ \Lambda, x, xx, xxx, \dots \}$

If $\Sigma = \{ 0, 1 \}$ then $\Sigma^* = \{ \Lambda, 0, 1, 00, 01, 10, 11, 000, 001, \dots \}$

If $\Sigma = \{ a, b, c \}$ then

$\Sigma^* = \{ \Lambda, a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, \dots \}$

Lexicographic order

- Notice that we listed the words in a language in size order (i.e., words of shortest length first), and then listed all the words of the same length alphabetically.
- This ordering is called lexicographic order, which we will usually follow.
- **The star in the closure notation is known as the Kleene star.**
- We can think of the Kleene star as an operation that makes, out of an alphabet, an infinite language (i.e., infinitely many words, each of finite length).

Kleene Closure

- Let us now generalize the use of the Kleene star operator to sets of words, not just sets of alphabet letters.
- Definition: If S is a set of words, then S^* is the set of all finite strings formed by concatenating words from S , where any word may be used as often as we like, and where the null string Λ is also included.

Kleene Closure

- Example: If $S = \{ aa, b \}$
- $S^* = \{ \Lambda \text{ plus any word composed of factors of } aa \text{ and } b \}, \text{ or}$
- $S^* = \{ \Lambda \text{ plus any strings of } a\text{'s and } b\text{'s in which the } a\text{'s occur in even clumps} \}, \text{ or}$
- $S^* = \{ \Lambda, b, aa, bb, aab, baa, bbb, aaaa, aabb, baab, bbaa, bbbb, aaaab, aabaa, aabbb, baaaa, baabb, bbaab, bbbba, bbbbbb, \dots \}$
- Note that the string $aabaaab$ is not in S^* because it has a clump of a 's of length 3

How to prove a certain word is in the closure language S^*

- We must show how it can be **written as a concatenation** of words from the base set S .
- In the previous example, **to show that abaab is in S^*** , we can factor it as follows:

$$\text{abaab} = (\text{ab})(\text{a})(\text{ab})$$

- **These three factors are all in the set S** , therefore their concatenation is in S^* .
- Note **that the parentheses, ()**, are used for the sole purpose of demarcating the ends of factors.

Kleene Closure

- Observe that if the alphabet has no letters, then its closure is the language with the null string as its only word; that is

if $\Sigma = \emptyset$ (the empty set), then $\Sigma^* = \{ \Lambda \}$

- Also, observe that if the set S has the null string as its only word, then the closure language S^* also has the null string as its only word; that is

if $S = \{ \Lambda \}$, then $S^* = \{ \Lambda \}$

because $\Lambda\Lambda = \Lambda$.

- Hence, the Kleene closure always produces an infinite language unless the underlying set is one of the two cases above.

Kleene Closure of different sets

- The Kleene closure of two different sets can end up being the same language.
- Example: Consider two sets of words

$$S = \{ a, b, ab \} \text{ and } T = \{ a, b, bb \}$$

- Then, ***both S^* and T^* are languages of all strings of a's and b's*** since any string of a's and b's can be factored into syllables of (a) or (b), both of which are in S and T.

Positive Closure

- If we wish to modify the concept of closure to refer only the concatenation of some (not zero) strings from a set S , **we use the notation $+$ instead of $*$.**
- This “plus operation” is called positive closure.
- Example

if $\Sigma = \{ x \}$ then $\Sigma^+ = \{ x, xx, xxx, \dots \}$

Positive Closure

- Observe that:
- If S is a language that does not contain Λ , then S^+ is the language S^* without the null word Λ .
- If **S is a language that does contain Λ , then $S^+ = S^*$**
- Likewise,
- **if Σ is an alphabet, then Σ^+ is Σ^* without the word Λ .**

S^{**}

- What happens if we apply the closure operator twice?
- We start with a set of words S and form its closure S^*
- We then start with the set S^* and try to form its closure, which we denote as $(S^*)^*$ or S^{**}

S^{**}

- Theorem 1:

For any set S of strings, we have $S^* = S^{**}$

Before we prove the theorem, recall from Set Theory that

- $A = B$ if A is a subset of B and B is a subset of A
- A is a subset of B if for all x in A , x is also in B

Proof

- Let us first **prove that S^{**} is a subset of S^* :**

Every word in S^{**} is made up of factors from S^* . Every factor from S^* is made up of factors from S . Hence, every word from S^{**} is made up of factors from S . Therefore, every word in S^{**} is also a word in S^* . This implies that S^{**} is a subset of S^* .

- Let us now prove that S^* is a subset of S^{**} :
- In general, it is true that for any set A , we have a subset A^* , because in A^* we can choose as a word any factor from A . So if we consider A to be our set S^* then S^* is a subset of S^{**}

Together, these two inclusions prove that $S^* = S^{}$.**

Recursive Definition

Recursive definition of languages

A recursive definition is characteristically a three-step process.

1. First, **we specify some basic objects** in the set.
2. Second, we give rules **for constructing more objects in the set** from the ones we already know.
3. Third, we **declare that no objects except those constructed** in this way are allowed in the set.

Recursive definition of languages

- The following three steps are used in recursive definition
 - Some basic words are specified in the language
 - Rules for constructing more words are defined in the language
 - No strings except those constructed in above, are allowed to be in the language

Example : Defining language of EVEN

Step 1:

- 2 is in EVEN.

Step 2:

- If x is in EVEN then $x+2$ and $x-2$ are also in EVEN.
- If x and y are in EVEN then so are $x+y$, $x-y$ and $x*y$.

Step 3:

- No strings except those constructed in above, are allowed to be in EVEN.

Example:

Defining the language PALINDROME, defined over $\Sigma = \{a,b\}$

Step 1:

- λ , a and b are in PALINDROME

Step 2:

- if x is palindrome then axa, bxb, xx are also be palindrome,

Step 3:

- No strings except those constructed in above, are allowed to be in palindrome

Example:

Defining the language $\{a^n b^n\}$, $n=1,2,3,\dots$, of strings defined over $\Sigma=\{a,b\}$

Step 1:

ab is in $\{a^n b^n\}$

Step 2:

if x is in $\{a^n b^n\}$, then $a x b$ is in $\{a^n b^n\}$

Step 3:

No strings except those constructed in above, are allowed to be in $\{a^n b^n\}$

Example:

Defining the language L , of strings ending in a , defined over $\Sigma = \{a, b\}$

Step 1:

a is in L

Step 2:

if x is in L then $s(x)$ is also in L , where s belongs to Σ^*

Step 3:

No strings except those constructed in above, are allowed to be in L

Example:

Defining the language L , of strings beginning and ending in same letters, defined over $\Sigma = \{a, b\}$

Step 1:

a and b are in L

Step 2:

$(a)s(a)$ and $(b)s(b)$ are also in L , where s belongs to Σ^*

Step 3:

No strings except those constructed in above, are allowed to be in L

Defining Polynomial

- A polynomial is a finite sum of terms each of which is of the form a real number times a power of x (that may be $x^0 = 1$).
- The set POLYNOMIAL is defined by **these four rules:**

Rules

- Rule 1:
 - Any number is in POLYNOMIAL.
- Rule 2:
 - The variable x is in POLYNOMIAL.
- Rule 3:
 - If p and q are in POLYNOMIAL, then so are $p + q$ and (p) and pq .
- Rule 4:
 - POLYNOMIAL contains only those things which can be created by the three rules above

Example

- Show that $3x^2 + 7x - 9$ is in POLYNOMIAL.

Solution

- **By Rule 1:** 3 is in POLYNOMIAL
- **By Rule 2:** x is in POLYNOMIAL
- **By Rule 3:** (3)(x) is in POLYNOMIAL, call-it 3x
- **By Rule 3:** (3x)(x) is in POLYNOMIAL, call it 3x²
- **By Rule 1:** 7 is in POLYNOMIAL
- **By Rule 3:** (7)(x) is in POLYNOMIAL
- **By Rule 3:** 3x' + 7x is in POLYNOMIAL
- **By Rule 1:** -9 is in POLYNOMIAL
- **By Rule 3:**
 $3x^2 + 7x + (-9) = 3x^2 + 7x - 9$ is in POLYNOMIAL.

Arithmetic Expression

- Suppose we ask ourselves what constitutes a valid arithmetic expression, or AE for short.
- The alphabet for this language is
- $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (,)\}$

Arithmetic Expression

- Obviously, the following expressions are not valid:

$(3 + 5) + 6)$ $2(/8 + 9)$ $(3 + (4-)8)$

- The first contains unbalanced parentheses; the second contains the forbidden substring (/; the third contains the forbidden substring -).
- Are there more rules? The substrings // and */ are also forbidden.
- Are there still more?
- The most natural way of defining a valid AE is by using a **recursive definition**, rather than a long list of forbidden substrings.

Recursive Definition of AE

- **Rule 1:** Any number (positive, negative, or zero) is in AE.
- **Rule 2:** If x is in AE, then so are
 - (i) x
 - (ii) $-x$ (provided that x does not already start with a minus sign)
- **Rule 3:** If x and y are in AE, then so are
 - (i) $x + y$ (if the first symbol in y is not $+$ or $-$)
 - (ii) $x - y$ (if the first symbol in y is not $+$ or $-$)
 - (iii) $x * y$
 - (iv) x / y
 - (v) $x ** y$ (our notation for exponentiation)

Arithmetic Expression

- The above definition is the most natural, because **it is the method we use to recognize valid arithmetic expressions** in real life.
- For instance, we wish to determine if the following expression is valid:

$$(2 + 4) * (7 * (9 - 3)/4)/4 * (2 + 8) - 1$$

- We do not really scan over the string, looking for **forbidden substrings** or **count the parentheses**.
- We actually imagine the expression in our mind broken down into components:
 - Is $(2 + 4)$ OK? Yes
 - Is $(9 - 3)$ OK? Yes
 - Is $7 * (9 - 3)/4$ OK? Yes, and so on.

Arithmetic Expression

- Note that the recursive definition of the set AE gives us the possibility of writing $8/4/2$, which is ambiguous, because it could mean $8/(4/2) = 4$ or $(8/4)/2 = 1$.
- However, the ambiguity of $8=4=2$ is a problem of *meaning*. There is no doubt that this string is a word in AE, only doubt about what it means.
- By applying Rule 2, we could always put enough parentheses to avoid such a confusion.
- The recursive definition of the set AE is useful for proving many theorems about arithmetic expressions, as we shall see in the next few slides.

Theorem

- **An arithmetic expression cannot contain the character \$.**

Theorem Proof

- This character is not part of any number, so it cannot be introduced into an AE by Rule 1.
- If the character string x does not contain the character $\$,$ then neither do the string (x) and $-x$. So, the character $\$$ cannot be introduced into an AE by Rule 2.
- If neither x nor y contains the character $\$,$ then neither do any of the expressions defined in Rule 3.
- Therefore, the character $\$$ can never get into an AE.

Propositional Calculus

- Propositional calculus (or sentential calculus) is a branch of symbolic logic that we shall be interested in.
- The version we define here uses only negation ($\neg$) and implication ($\rightarrow$), together with the phrase variables.
- The alphabet for this language is

$$\Sigma = \{\neg, \rightarrow, (,), a, b, c, d, \dots\}$$

- A valid expression in this language is called WFF (well-form formula).

Propositional Calculus

- The rules for forming WFFs are:
 - **Rule 1:** Any single Latin letter is a WFF, for instance a, b, c, ...
 - **Rule 2:** If p is a WFF, then so are (p) and $\neg p$.
 - **Rule 3:** If p and q are WFFs, then so is $p \rightarrow q$.
- Can you show that $p \rightarrow ((p \rightarrow p) \rightarrow q)$ is a WFF?
- Can you show that the following are NOT WFFs?
 - $p \rightarrow$
 - $\rightarrow p$
 - $p) \rightarrow p($

Practice Theorem

- 1. No arithmetic expression can begin or end with the symbol /**
- 2. No arithmetic expression can contain the substring //**